

Präsentiert von: Marius Graumann, Exir Forouzani, Gaby Fotio, Tom Estermann

Teammitglieder

Projektleiter

Marius Graumann

- Verantwortlich für Gesamtprojekterfolg

- Vorbereitung der Treffen mit dem Auftraggeber

- Koordination von Meetings

Lead Requirements Engineer

Tom Estermann

- Verantwortlich für das Pflichtenheft
  - Klärung der Stakeholderanforderungen
  - Ergänzung der Stakeholderanforderungen

Lead Software-Architekt

Exir Forouzani

- Verantwortlich für Software Entwurf
- Implementierung der Software
- Stellvertretender Projektmanager

Lead Testmanager

Gaby Fotio

- Verantwortlich für Testkonzept
- Durchführung von Tests
- Dokumentation von Tests

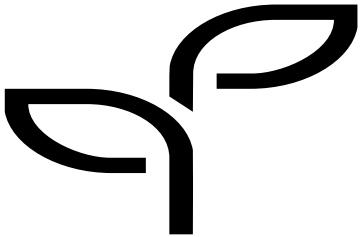
Tools

Tools









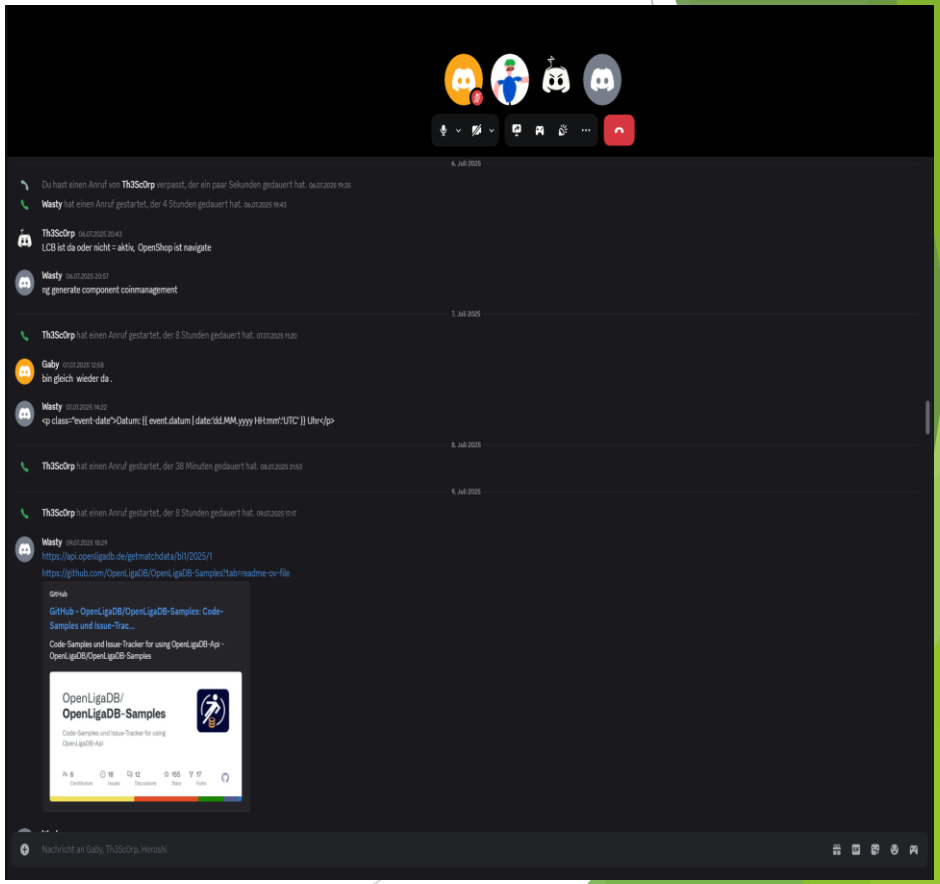




Discord

Meetings und Screensharing

Schneller Dateiaustausch

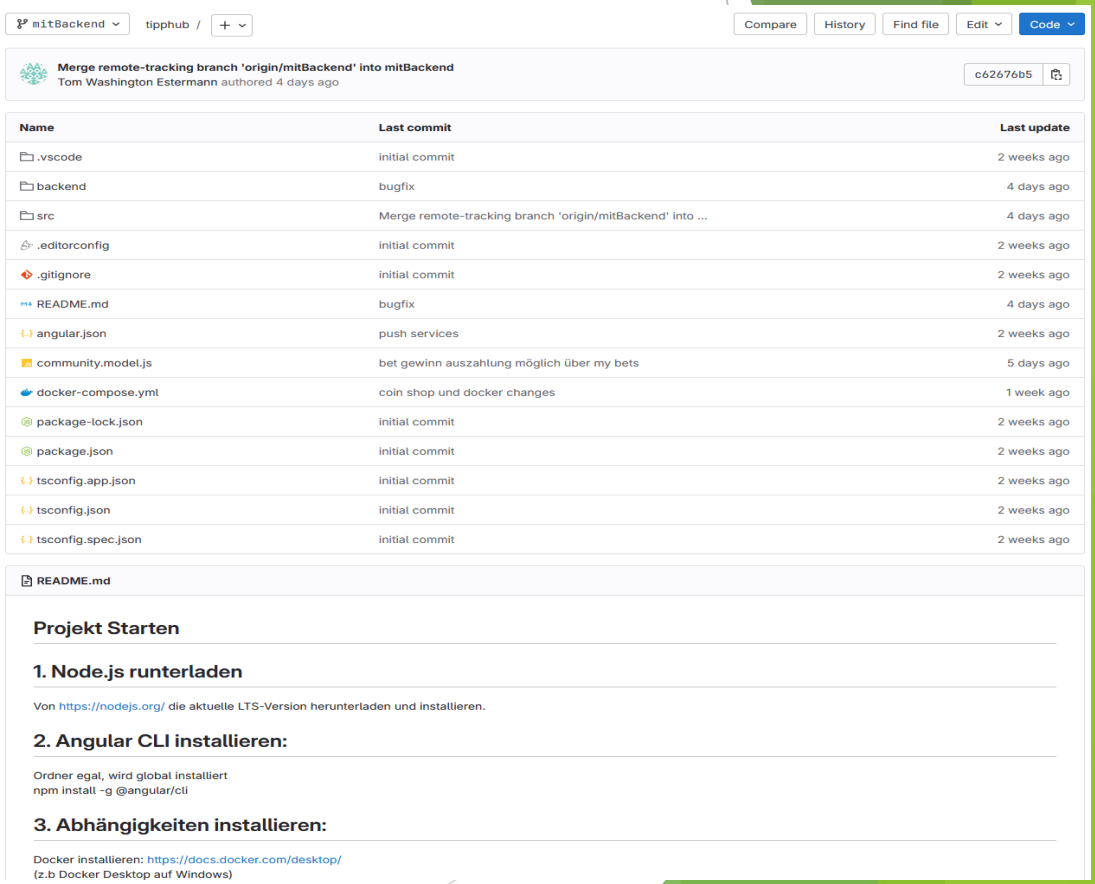


Git



Versionsverwaltung des Projektcodes

Auch für LaTeX und PlantUML

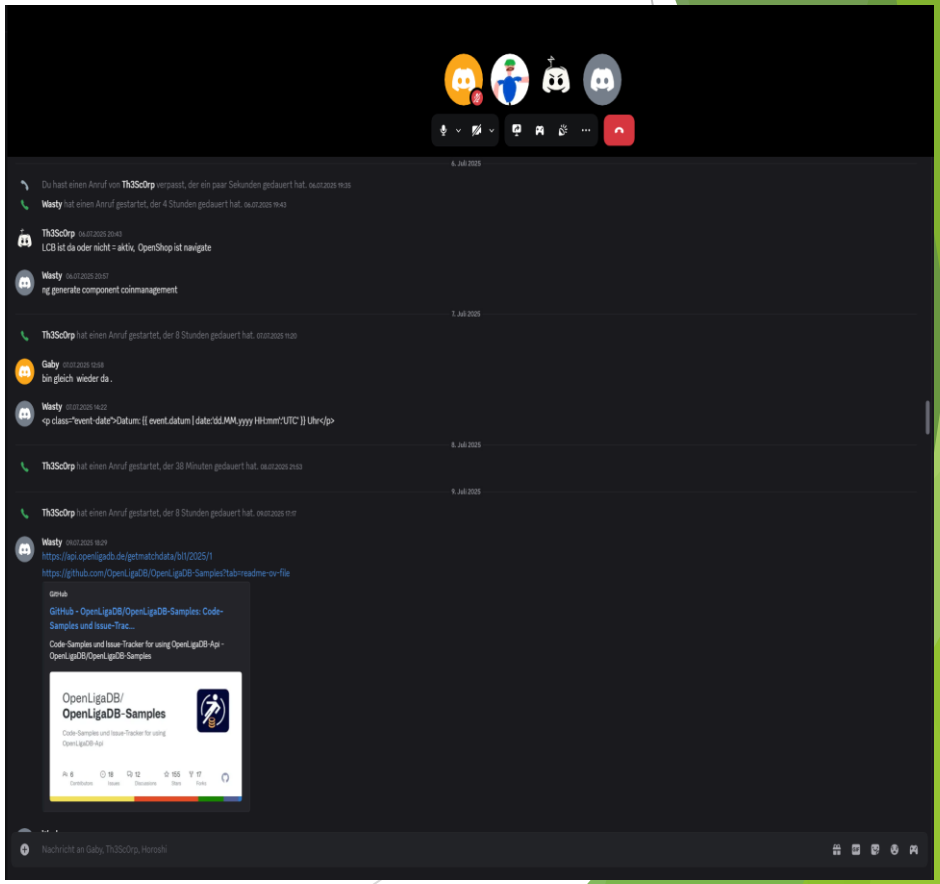


LaTeX

TEX

Dokumentenerstellung

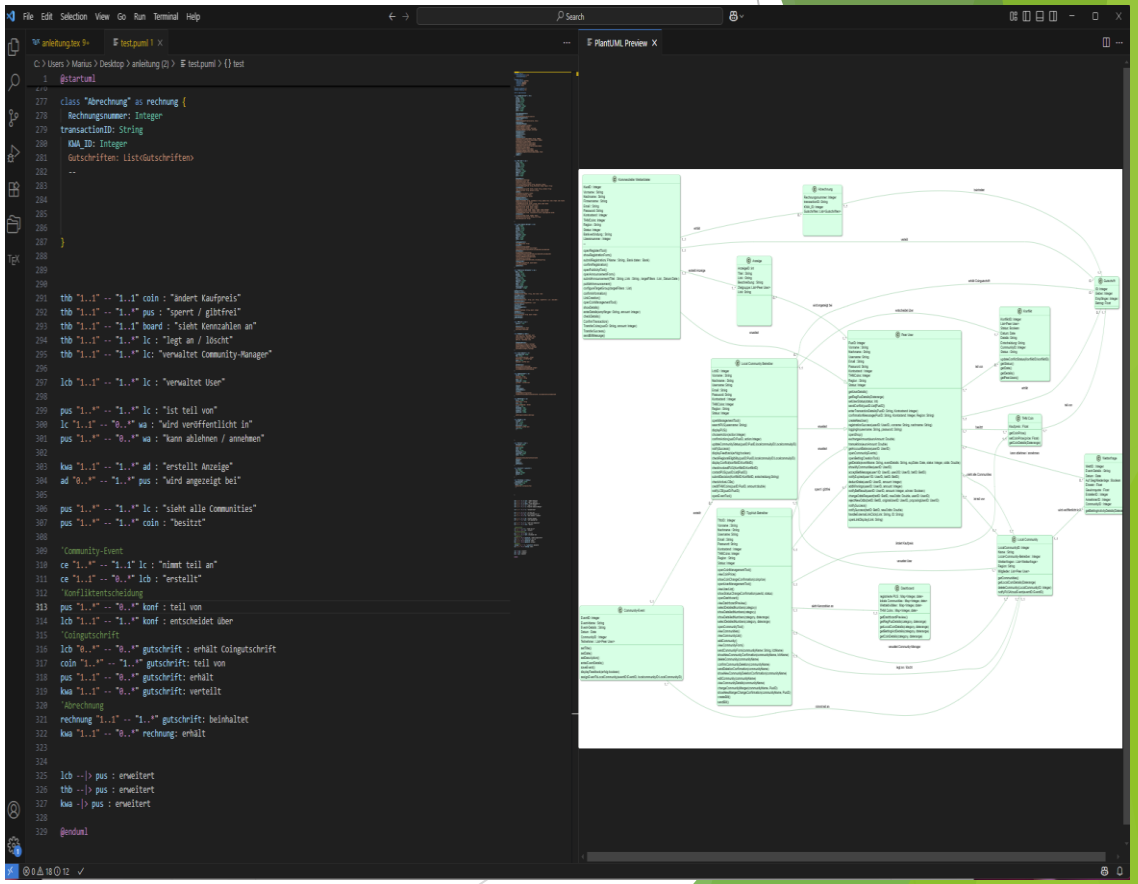
Leichtes Coworking



PlantUML

Erstellung von Diagrammen

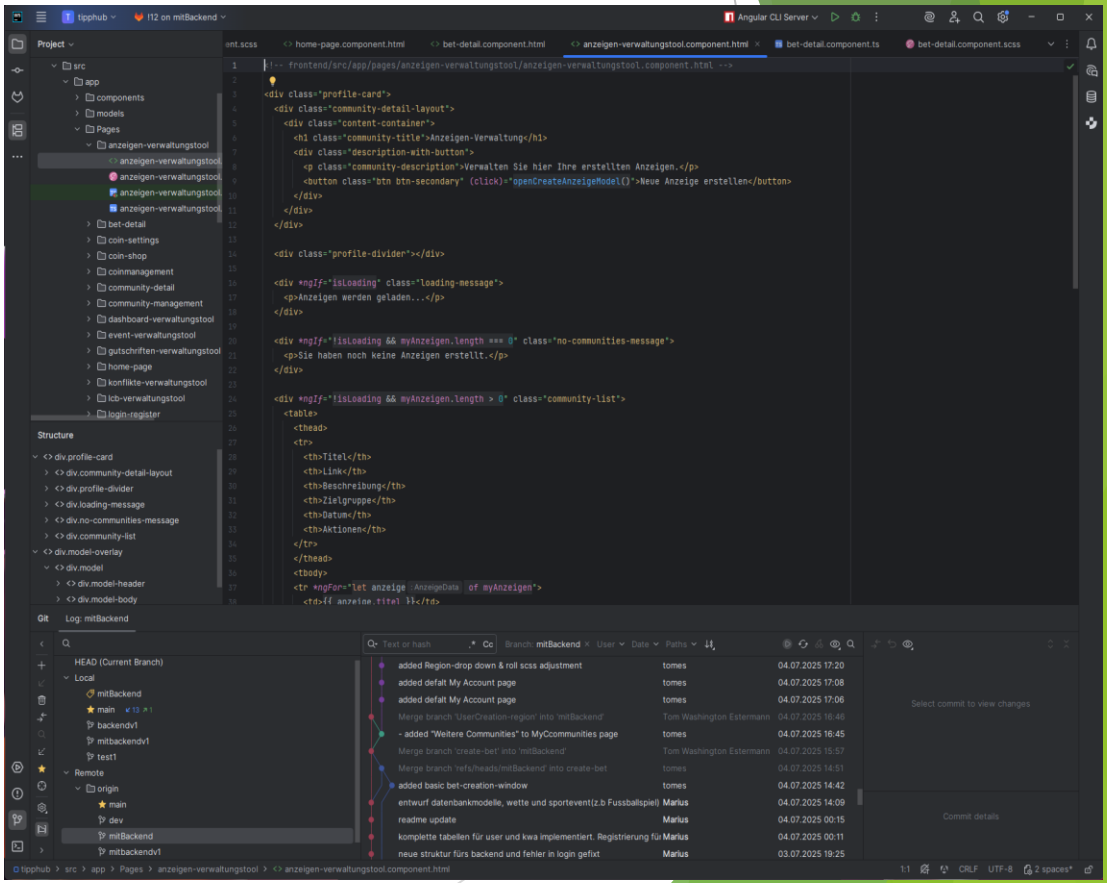
Leichtes Coworking



WebStorm

Entwicklungsumgebung

Leichte Nutzung von Git



i

- irgendwas
  - Irgendwas
  - irgendwas

Pflichtenheft (OOA)

Stakeholder Anforderungen & Use Cases

Stakeholder Anforderungen

StR.THB.3	Der THB möchte die Möglichkeit haben, Peer User auf der Plattform zu sperren und ggf. wieder freizugeben.
StR.LCB.5	LCB sollen in der Lage sein, spezielle Event der lokalen Community anzulegen, worüber alle PUS der lokalen Community informiert werden.

Use Case: Peer User sperren

Use Case (Name)	**UC-2 Peer User sperren**
Primärer Akteur	TippHub Betreiber
Weitere Akteure	Peer User
Stakeholder-Ziele	StR.THB.3
Auslösendes Ereignis	Wunsch einen Peer User zu sperren
Beschreibung	Der Betreiber hat einen Liste von Peer Usern, ggf. mit einer Suchemaske. Mit einem Dropdown Menü kann er den Status eines Peer Users ändern. Der Betreiber kann den User dann auf gesperrt setzen.
Detailbeschreibung	7.1.2
Beziehung zu anderen Use Cases	

Use Case: Peer User freigeben

Use Case (Name)	**UC-3: Peer User freigeben**
Beschreibung der Anforderung	
Primärer Akteur	TippHub Betreiber
Weitere Akteure	Peer User
Stakeholder-Ziele	StR.THB.3
Auslösendes Ereignis	Wunsch einen Peer User zu sperren
Beschreibung	Der Betreiber hat einen Liste von Peer Usern, ggf. mit einer Suchemaske. Mit einem Dropdown Menü kann er den Status eines Peer Users ändern. Der Betreiber kann den User dann auf freigegeben setzen.
Detailbeschreibung	7.1.3
Beziehung zu anderen Use Cases	

Use Case: Lokales Community-Event anlegen

Use Case (Name)	**UC-13: Lokales Community-Event anlegen**
Primärer Akteur	Local Community Betreiber
Weitere Akteure	Peer User
Stakeholder-Ziele	StR.LCB.5
Auslösendes Ereignis	LCB entscheidet, ein neues Event für die lokale Community zu planen.

Beschreibung	Der LCB kann über ein Eingabeformular ein neues Event anlegen. Dabei gibt er Details des Events an und nach dem Speichern werden automatisch alle PUS der lokalen Community über das neue Event informiert.
Detailbeschreibung	7.2.5
Beziehung zu anderen Use Cases	

Detailansicht der Usecases

Use Case: Peer User sperren

Vorbedingungen:	1. THB hat sich im TippHub-System angemeldet 2. THB hat die Seite: Überverwaltung" geöffnet
Nachbedingung:	Der User wurde im System gesperrt
Hauptszenario:	1. Das TippHub-System eine Liste der aktuellen User und ihren Status. 2. Der THB sucht den User manuell, oder über eine Suchleiste. 3. Der THB klickt auf das Dropdownmenü neben dem User. 4. Der THB wählt sperren aus. 5. Das TippHub-System bestätigt die Sperrung des Users mit einer Einblendung.
Ausnahmeszenarien:	

Use Case: Peer User freigeben

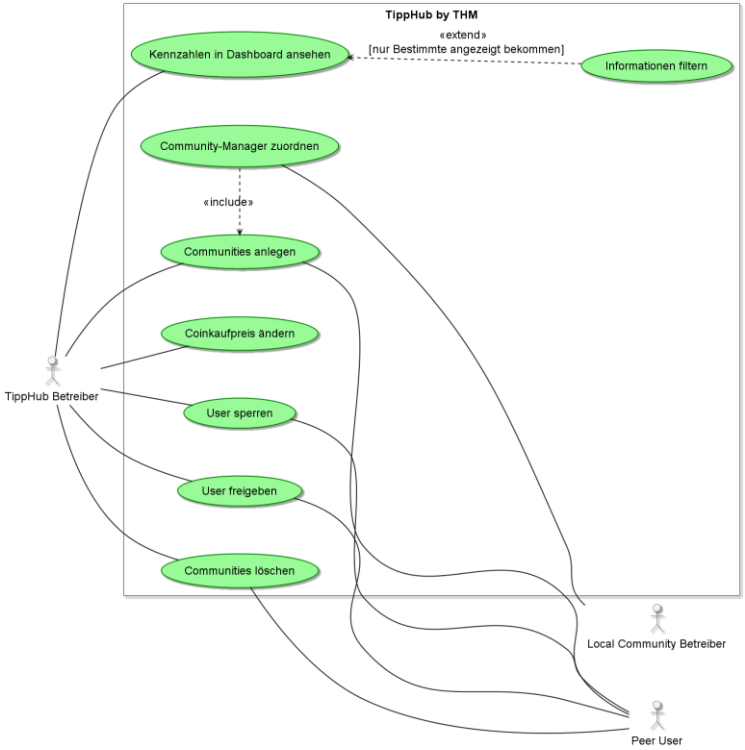
Vorbedingungen:	1. THB hat sich im TippHub-System angemeldet 2. THB hat die Seite: Üserverwaltung"geöffnet
Nachbedingung:	Der User wurde im System freigegeben
Hauptszenario:	1. Das TippHub-System eine Liste der aktuellen User und ihren Status. 2. Der THB sucht den User manuell, oder über eine Suchleiste. 3. Der THB klickt auf das Dropdownmenü neben dem User. 4. Der THB wählt freigeben aus. 5. Das TippHub-System bestätigt das Freigeben des Users mit einer Einblendung.
Ausnahmeszenarien:	

Use Case: Lokales Community-Event anlegen

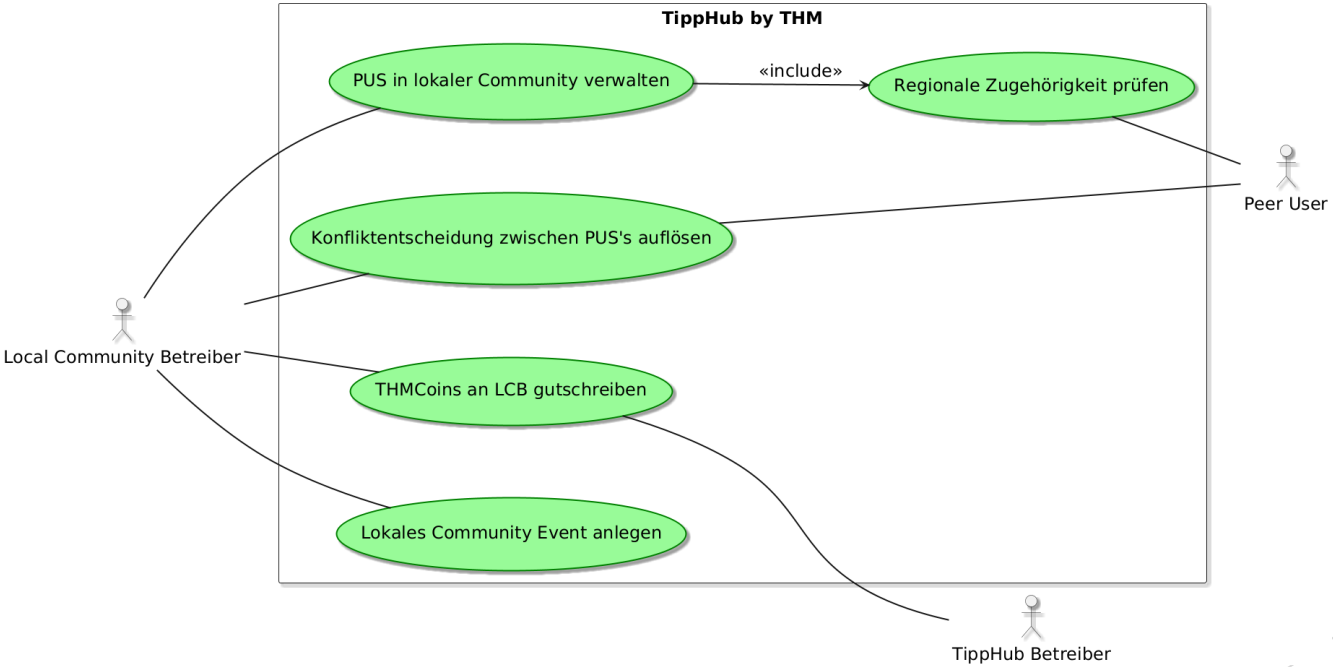
Vorbedingungen:	1. Der LCB hat sich im TippHub-System angemeldet 2. Die lokale Community existiert im System
Nachbedingung:	1. Ein Event wurde angelegt 2. Alle PUSs in der lokalen Community wurden benachrichtigt
Hauptszenario:	1. LCB öffnet das Event-Verwaltungstool 2. LCB gibt die Eventdetails ein 3. LCB speichert das Event 4. System legt das Event an und verknüpft es mit der lokalen Community 5. System informiert automatisch alle PUs der Community über das Event
Ausnahmeszenarien:	3a. System wird aufmerksam, dass die angegebenen Eventdetails falsch oder nicht vorhanden sind 3a1. Das System zeigt eine Fehlermeldung: Eventdetails sind ungültig → weiter mit 4 im Hauptszenario

Graphische Darstellung

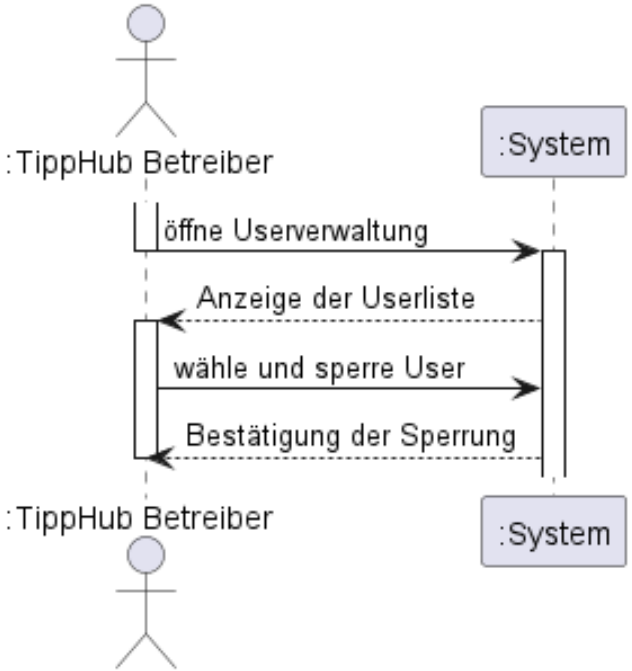
Use-Case-Diagramm: THB

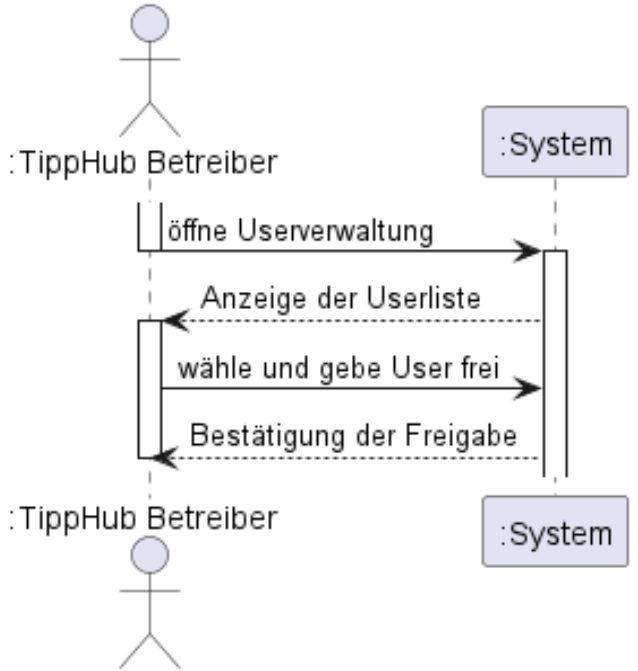


Use-Case-Diagramm: LCB

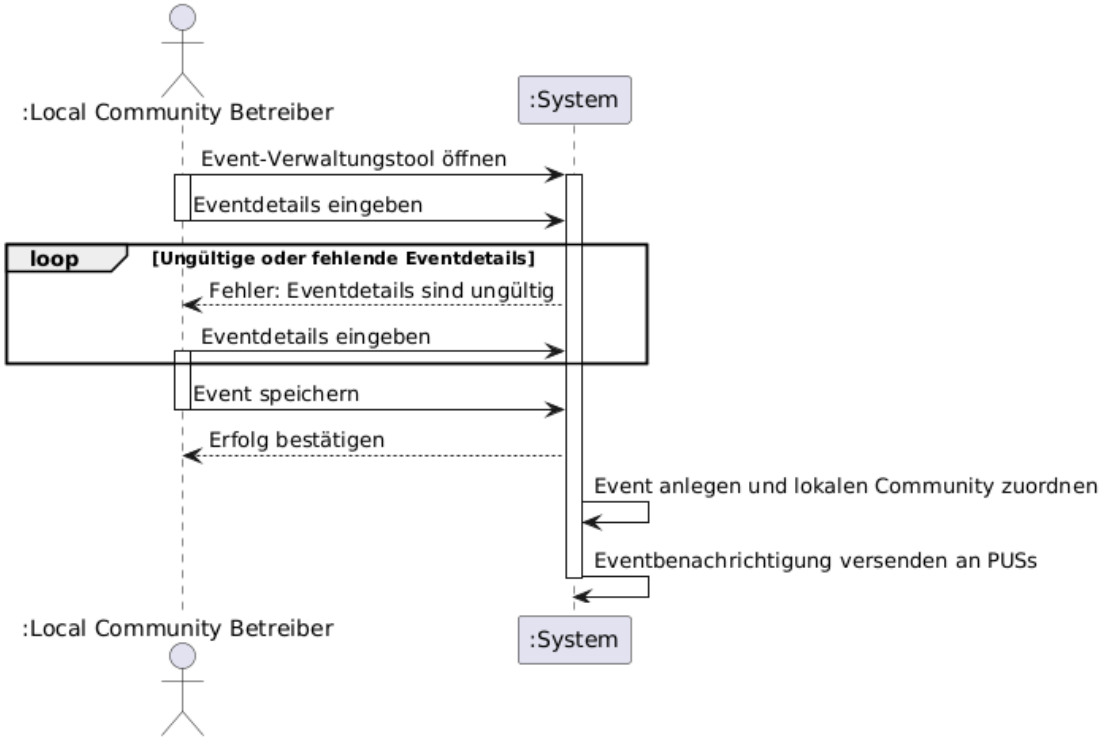


Sequenzdiagramm: User sperren/freigeben





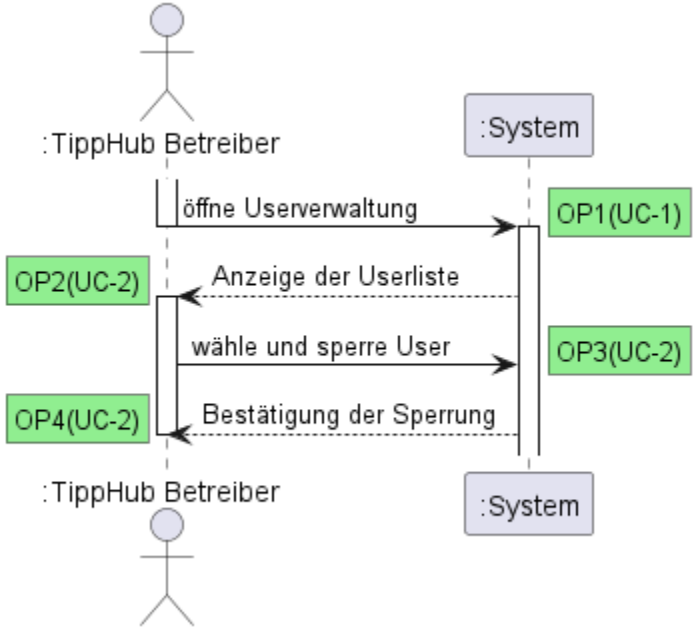
Sequenzdiagramm: Lokales Community-Event anlegen



Softwareentwurf (OOD)

Operationelle Verträge

OP: Peer User sperren



Operation OP3(UC-2): setUserStatus

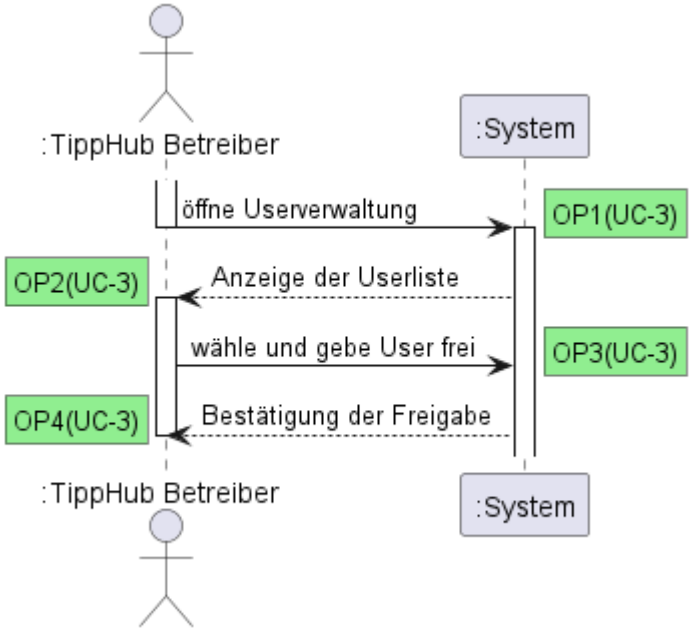
Operation: setUserStatus(status: Integer)
Querverweis: UC-2: User sperren
Vorbedingungen:

- Das User-Verwaltungstool ist geöffnet.

Nachbedingungen:

- Das Attribut, User.status, wurde auf 1 gesetzt.

OP: Peer User freigeben



Operation OP3(UC-3): setUserStatus

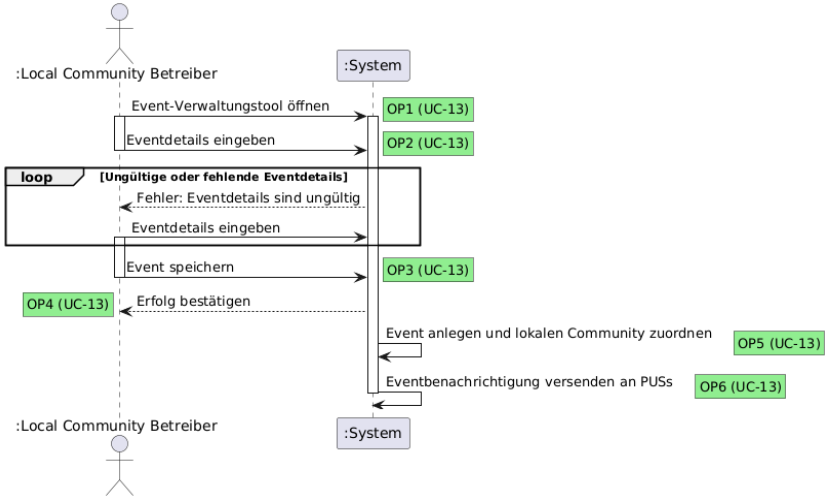
Operation: setUserStatus(status: Integer)
Querverweis: UC-3: User freigeben
Vorbedingungen:

- Das User-Verwaltungstool ist geöffnet.

Nachbedingungen:

- Das Attribut, User.status, wurde auf 0 gesetzt.

OP: Lokales Community-Event anlegen



Operation OP3(UC-13): saveEvent

Operation: saveEvent()
Querverweis: UC-13: Lokales Community-Event anlegen
Vorbedingungen:

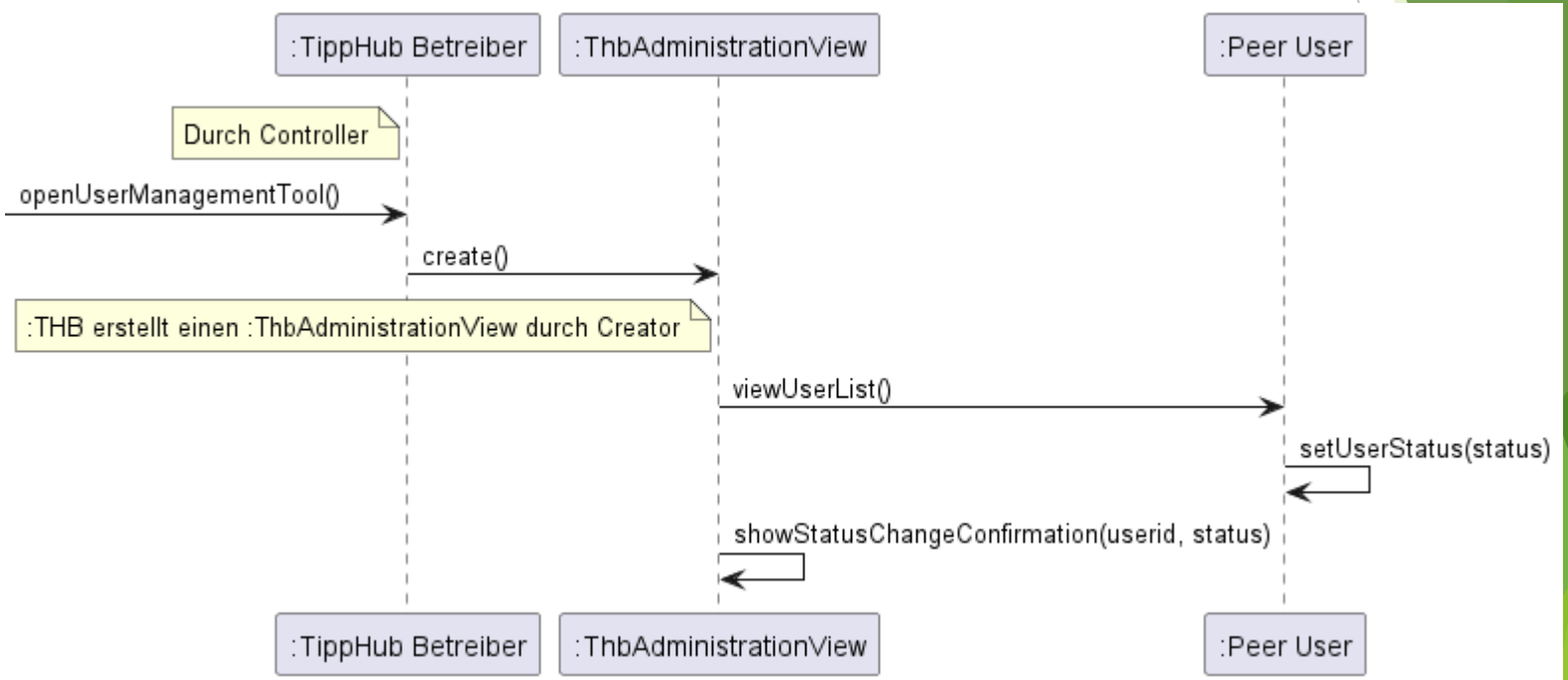
- Gültige Eventdetails wurden eingegeben.

Nachbedingungen:

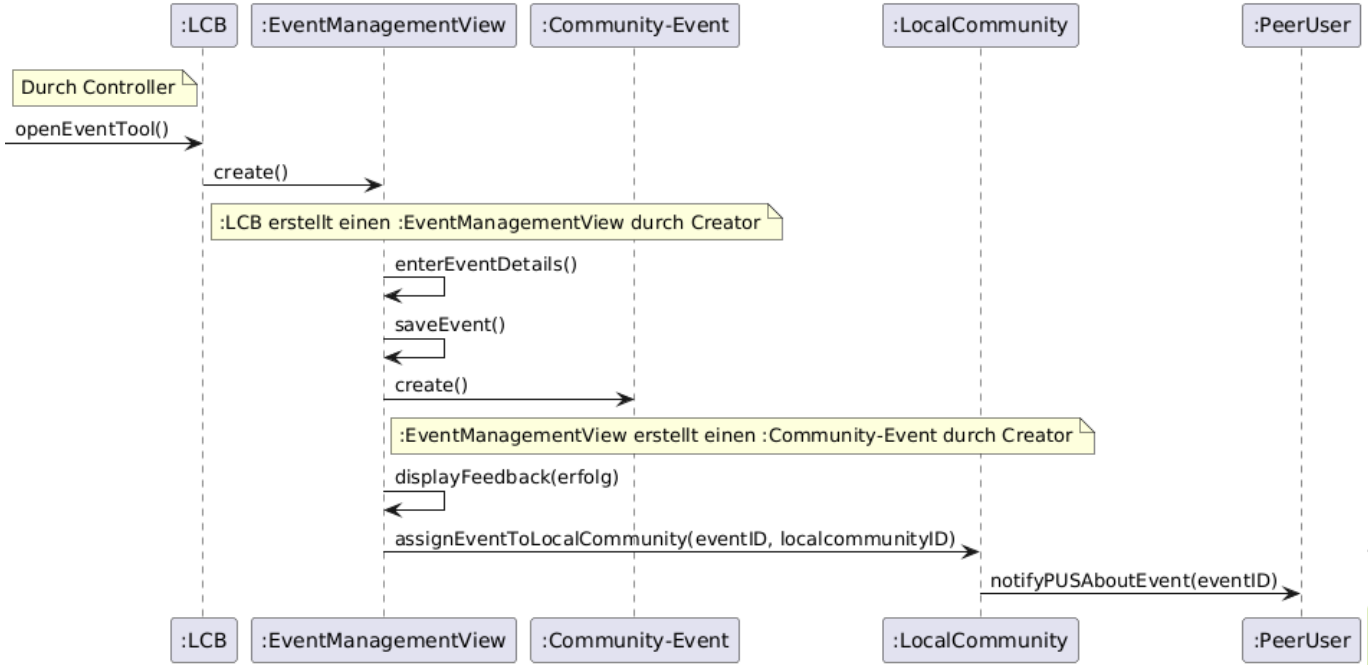
- Intern wurden Setter-Methoden auf dem eventmanagementview Objekt aufgerufen
  - `setTitle()`
  - `setDate()`
  - `setDescription()`
- Die eingegebenen Eventdetails wurden validiert.

Interaktionsdiagramme

Interaktionsdiagramm: Peer User sperren/ freigeben



Interaktionsdiagramm: Lokales Community-Event anlegen



Testkonzept

Testkonzept

Lokales Community Event anlegen

Prüfen, ob das Erstellen eines Events im gesamten System – von der Eingabe bis zur Anzeige – vollständig, korrekt und benutzbar funktioniert.

Peer User freigeben

Test des Zusammenspiels zwischen Benutzerverwaltung, Statusverwaltung, Sicherheitsrichtlinien und ggf. Benachrichtigungssystemen.

Peer User sperren

Sicherstellen, dass das Sperren eines Nutzers korrekt funktioniert – ohne unbeabsichtigte Seiteneffekte.

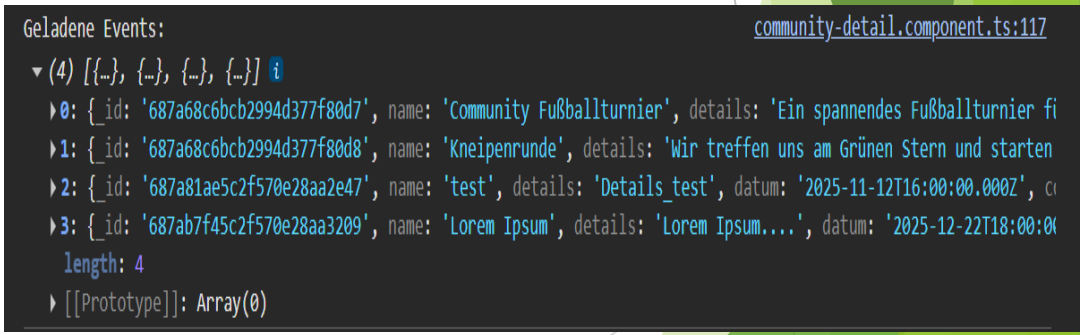
Lokales Community Event anlegen

2.5 Lokales Community-Event anlegen (UC-13)

2.5.1 Testfall: UC-13 – HS: Lokales Community-Event anlegen

Angaben für die Durchführung eines Tests	LCB legt erfolgreich ein Event für seine lokale Community an
Vorbedingungen	• LCB ist im TippHub-System angemeldet • Die lokale Community des LCB existiert im System
Nachbedingungen	• Das Event wurde erfolgreich im System angelegt • Alle PUSs der lokalen Community wurden benachrichtigt
Eingabewerte	• Titel:1-Jubiläum • Datum: 23.06.2025 • Beschreibung: Feier zum einjährigen der lokalen Community.
Erwartete Ausgabewerte	• Benachrichtigung an alle PUSs der Community
Prüfanweisungen	• LCB meldet sich im System an • LCB öffnet das Event-Verwaltungstool • LCB trägt vollständige Eventdetails ein und speichert • Überprüfung, ob Event erfolgreich gespeichert wurde • Überprüfung, ob Benachrichtigung an alle PUSs erfolgt ist





EVENTS

Community Fußballturnier
20.08.2025 16:00
0 Teilnehmer

Kneipenrunde
23.08.2025 22:00
1 Teilnehmer

test
12.11.2025 17:00
0 Teilnehmer

Lorem Ipsum
22.12.2025 19:00
0 Teilnehmer

```
Geladene Events:                                                    community-detail.component.ts:117
▼ (4) [{…}, {…}, {…}, {…}]
  ▶ 0: {_id: '687a68c6bcb2994d377f80d7', name: 'Community Fußballturnier', details: 'Ein spannendes Fußballturnier fi
  ▶ 1: {_id: '687a68c6bcb2994d377f80d8', name: 'Kneipenrunde', details: 'Wir treffen uns am Grünen Stern und starten
  ▶ 2: {_id: '687a81ae5c2f570e28aa2e47', name: 'test', details: 'Details_test', datum: '2025-11-12T16:00:00.000Z', c
  ▶ 3: {_id: '687ab7f45c2f570e28aa3209', name: 'Lorem Ipsum', details: 'Lorem Ipsum....', datum: '2025-12-22T18:00:0
    length: 4
  ▶ [[Prototype]]: Array(0)
```

Peer User sperren/ freigeben

1.3 User freigeben (UC-3)

1.3.1 Testfall: UC-3 – HS: User entsperren

Angaben für die Durchführung eines Tests	THB gibt einen gesperrten User im System frei
Vorbedingungen	• THB ist erfolgreich im TippHub-System angemeldet • Die Seite "Userverwaltung" ist geöffnet • Der freizugebende User existiert und ist gesperrt
Nachbedingungen	• Der User wurde im System freigegeben • THB erhält eine Bestätigung der Freigabe
Eingabewerte	• Name des Users: Max Mustermann (Beispiel) • Aktion: "Freigeben"
Erwartete Ausgabewerte	• Liste der User mit Status wird angezeigt • Bestätigungsmeldung: „User wurde freigegeben.“
Prüfanweisungen	• Als THB anmelden und die Seite "Userverwaltung" öffnen • Nach einem existierenden, gesperrten User suchen • Auf das Dropdownmenü neben dem User klicken • "Freigeben" auswählen • Prüfen, ob die Bestätigungsmeldung korrekt erscheint und der User-Status aktualisiert wurde

1.2 User sperren (UC-2)

1.2.1 Testfall: UC-2 – HS: User blockieren

Angaben für die Durchführung eines Tests	THB sperrt einen User im System
Vorbedingungen	• THB ist erfolgreich im TippHub-System angemeldet • Die Seite "Userverwaltung" ist geöffnet • Der zu sperrende User existiert und ist nicht bereits gesperrt
Nachbedingungen	• Der User wurde im System gesperrt • THB erhält eine Bestätigung der Sperrung
Eingabewerte	• Name des Users: Max Mustermann (Beispiel) • Aktion: "Sperren"
Erwartete Ausgabewerte	• Liste der User mit Status wird angezeigt • Bestätigungsmeldung: „User wurde gesperrt.“
Prüfanweisungen	• Als THB anmelden und die Seite "Userverwaltung" öffnen • Nach einem existierenden, nicht gesperrten User suchen • Auf das Dropdownmenü neben dem User klicken • "Sperren" auswählen • Prüfen, ob die Bestätigungsmeldung korrekt erscheint und der User-Status aktualisiert wurde

Benutzerdaten geladen: user-management.component.ts:30

```
▼ Array(6) i
  ▼ 0:
      email: "user1@example.com"
      kontostand: 100
      nachname: "Mustermann"
      region: "Berlin"
      role: "pus"
      status: "banned"
      thmcoins: 500
      username: "maxmuster"
      vorname: "Max"
      __v: 0
      _id: "687a68c6bcb2994d377f80c8"
```

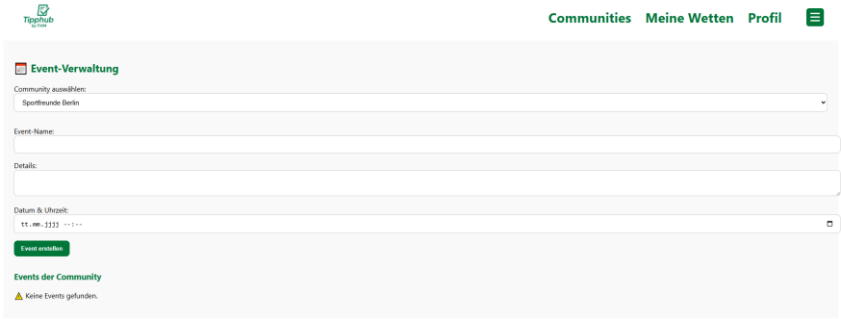
```
_id: ObjectId('687a68c6bcb2994d377f80c8')
email : "user1@example.com"
password : "password123"
vorname : "Max"
nachname : "Mustermann"
username : "maxmuster"
role : "pus"
kontostand : 100
thmcoins : 500
region : "Berlin"
status : "active"
__v : 0
```

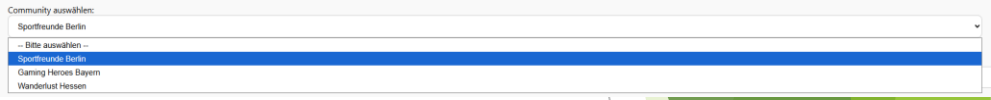
Benutzerstatus aktualisiert: user-management.component.ts:69

```
▼ {_id: '687a68c6bcb2994d377f80c8', email: 'user1@example.com', vorname: 'Max', nachname: 'Musterman
  n', username: 'maxmuster', …} i
    email: "user1@example.com"
    kontostand: 100
    nachname: "Mustermann"
    region: "Berlin"
    role: "pus"
    status: "active"
    thmcoins: 500
    username: "maxmuster"
    vorname: "Max"
    _id: "687a68c6bcb2994d377f80c8"
```

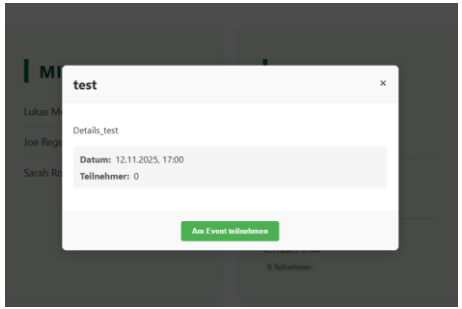
In der Anwendung (UI)

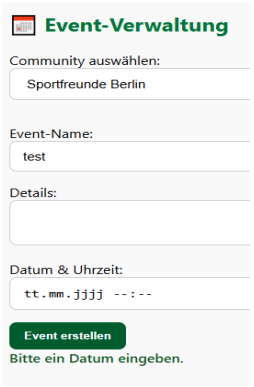
Lokales Community Event anlegen

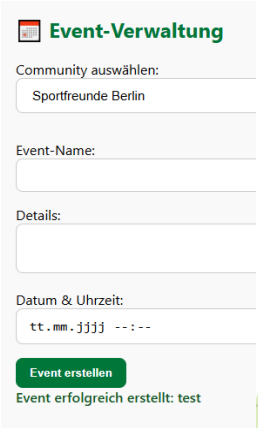


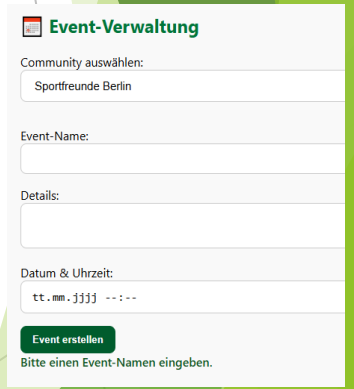




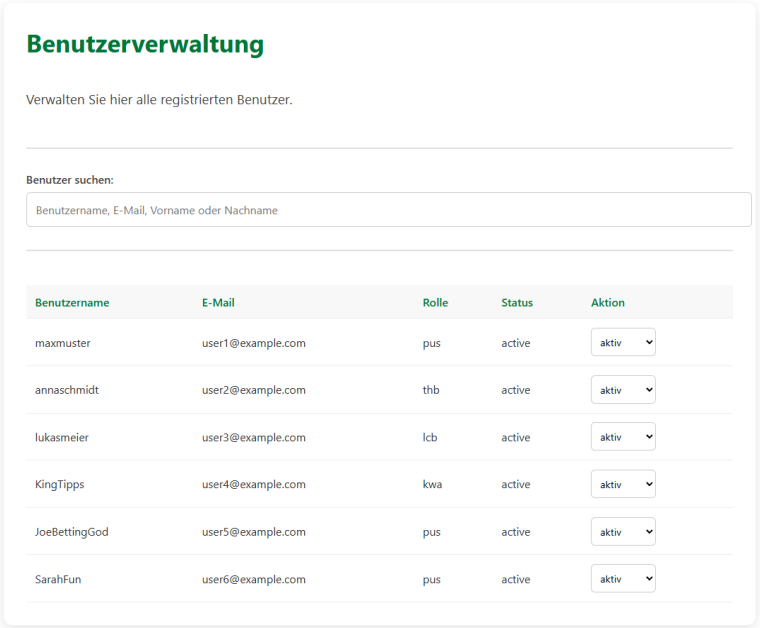


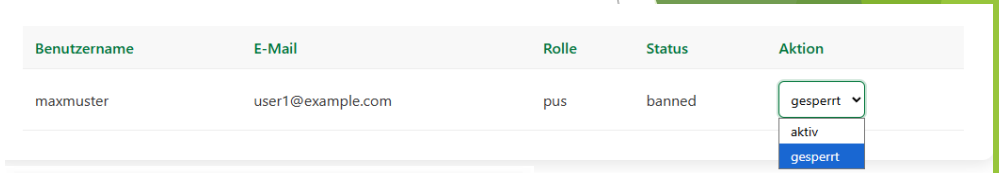




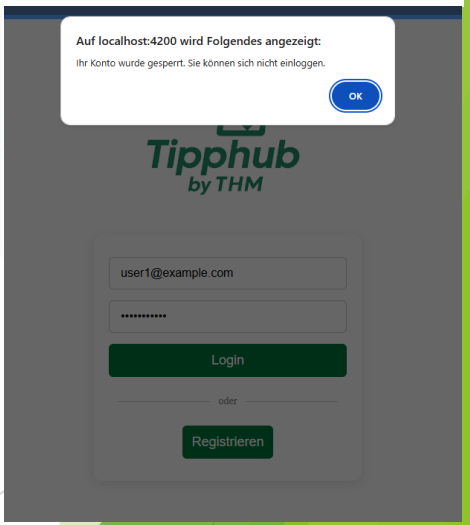


Peer User sperren/ freigeben



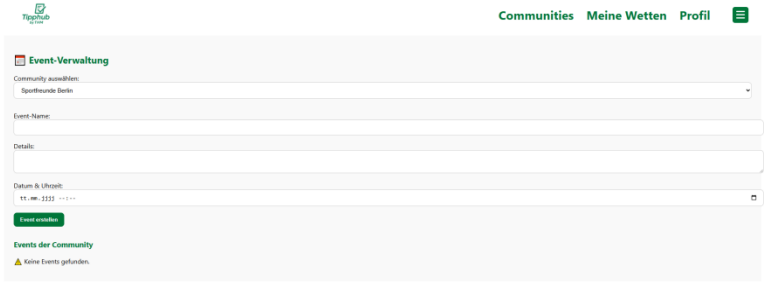






Code

Lokales Community Event anlegen




```
<label for="community">Community auswählen:</label>
<select id="community" [(ngModel)]="selectedCommunityId" (change)="onCommunityChange()" name="community">
  <option value="">-- Bitte auswählen --</option>
  <option *ngFor="let c : CommunityData of communities" [value]="c._id">{{ c.name }}</option>
</select>

<div *ngIf="selectedCommunityId">

  <form (ngSubmit)="saveEvent()" #eventForm : NgForm ="ngForm" style="...">
    <label for="name">Event-Name:</label>
    <input id="name" [(ngModel)]="newEvent.name" name="name" required />

    <label for="details">Details:</label>
    <textarea id="details" [(ngModel)]="newEvent.details" name="details"></textarea>

    <label for="datum">Datum & Uhrzeit:</label>
    <input
      id="datum"
      type="datetime-local"
      [(ngModel)]="newEvent.datum"
      name="datum"
      required
      min="2024-01-01T00:00"
      max="2100-12-31T23:59"
    />

    <button type="submit">Event erstellen</button>
  </form>
  <div *ngIf="showNotification" class="notification" style="...">
    {{ notificationMessage }}
  </div>
```

```
65    saveEvent(): void {  Show usages  Th3Sc0rp
66      if (!this.newEvent.communityId) {
67        this.displayFeedback( message: 'Bitte eine Community auswählen.');
68        return;
69      }
70      if (!this.newEvent.name.trim()) {
71        this.displayFeedback( message: 'Bitte einen Event-Namen eingeben.');
72        return;
73      }
74      if (!this.newEvent.datum) {
75        this.displayFeedback( message: 'Bitte ein Datum eingeben.');
76        return;
77      }
78
79      this.eventService.createEvent(this.newEvent).subscribe({
80        next: res : EventResponse  => {
81          this.displayFeedback( message: `Event erfolgreich erstellt: ${res.event.name}`);
82          this.onCommunityChange();
83          this.resetForm();
84        },
85        error: err => this.displayFeedback(err.message)
86      });
87    }
```

Peer User sperren/ freigeben

Benutzername	E-Mail	Rolle	Status	Aktion
maxmuster	user1@example.com	pus	banned	gesperrt

aktiv
gesperrt

```
<div *ngIf="!isLoading && filteredUsers.length > 0" class="user-list">
  <table>
    <thead>
    <tr>
      <th>Benutzername</th>
      <th>E-Mail</th>
      <th>Rolle</th>
      <th>Status</th>
      <th>Aktion</th>
    </tr>
    </thead>
    <tbody>
    <tr *ngFor="let user :UserData of filteredUsers">
      <td>{{ user.username }}</td>
      <td>{{ user.email }}</td>
      <td>{{ user.role }}</td>
      <td>{{ user.status }}</td>
      <td>
        <select [(ngModel)]="user.status" (change)="onStatusChange(user._id!, user.status)">
          <option value="active">aktiv</option>
          <option value="banned">gesperrt</option>
        </select>
      </td>
    </tr>
    </tbody>
  </table>
 </div>
</div>
```

```
54      onStatusChange(userId: string, newStatus: 'active' | 'banned'): void {  Show usages  Marius
55        if (!userId) {
56          console.error( message: 'Benutzer-ID fehlt für Statusänderung.');
57          alert( message: 'Fehler: Benutzer-ID fehlt.');
58          return;
59        }
60
61        this.userService.setUserStatus(userId, newStatus).subscribe({
62          next: (updatedUser: UserData) :void  => {
63            const index :number  = this.users.findIndex(u :UserData  => u._id === updatedUser._id);
64            if (index !== -1) {
65              this.users[index].status = updatedUser.status;
66              this.filterUsers();
67            }
68            alert( message: `Status für Benutzer "${updatedUser.username}" erfolgreich auf "${updatedUser.status}" aktualisiert.`);
69            console.log( message: 'Benutzerstatus aktualisiert:', updatedUser);
70          },
71          error: (error: HttpErrorResponse) :void  => {
72            console.error( message: 'Fehler beim Aktualisieren des Benutzerstatus:', error);
73            alert( message: 'Fehler beim Aktualisieren des Benutzerstatus. Bitte versuchen Sie es erneut.');
74          }
75        });
76      }
77    }
```

Danke für Ihre Aufmerksamkeit!